

ISIS-2111 PLE

Proyecto 2

Correcciones aplicadas a VeriLang — Retroalimentación TA (Entrega_2)

Amelia Serrano Arango - 202221556
Cristian David Ortiz Sanchez - 202311404

Mayo 2026

Resumen

Documento que detalla las correcciones realizadas sobre el código de la **Entrega_2** atendiendo la retroalimentación de la TA (Carla). La evaluación general de la entrega fue muy positiva, destacando que el archivo `ToAST.rsc` es excelente, utiliza correctamente el *pattern matching* de sintaxis concreta de Rascal, cubre todas las reglas, maneja los ocho operadores de comparación de forma individual e incluye un manejo adecuado de errores. Asimismo, se validó que el pipeline *parse ¿AST* funciona de extremo a extremo y que la jerarquía de precedencia de expresiones con recursión por la izquierda es correcta.

A partir de esta base sólida, el presente documento aborda las áreas principales de mejora identificadas por la TA. Se detalla la comparación entre la gramática formal documentada en la Entrega_2 y su versión corregida en la Entrega_3, mostrando cómo se resolvieron las limitaciones del lenguaje.

1. Correcciones en la Gramática Formal (documentos PDF)

Además de los cambios en el código Rascal, se realizaron correcciones en la **gramática formal** del lenguaje, documentadas en dos archivos:

Versión	Archivo
Antigua (Entrega_2)	Gramatica.pdf
Corregida (Entrega_3)	Correcciones_ple_project_1.pdf

1.1. Alfabeto — Letras mayúsculas y minúsculas

La versión antigua del alfabeto y sus reglas de producción se limitaban exclusivamente al uso de letras minúsculas:

- **Old:** El alfabeto solo definía caracteres de la `.a` a la `z`. La regla `Letter` englobaba directamente estas minúsculas.
- **New:** Se añadieron las letras mayúsculas de la `.A` a la `Z` al alfabeto. La regla léxica se reestructuró como `Letter ::= LowerLetter | UpperLetter`, definiendo explícitamente ambos conjuntos para permitir una mayor flexibilidad en la sintaxis.

1.2. Alfabeto — Operadores aritméticos y equivalencia

La versión antigua del alfabeto **no incluía** los operadores aritméticos ni el de equivalencia:

- **Old:** +, -, *, /, **, %, — ausentes
- **New:** Se añadieron al alfabeto todos los operadores:

```
+ - * / ** % < > <= >= <> = -> => ≡ and or neg in
```

1.3. No terminales — Expresiones aritméticas

La gramática antigua carecía de los no terminales para la jerarquía aritmética:

- **Old:** $N = \{ \dots, \text{EqExpr}, \text{UnaryExpr}, \text{Atom}, \dots \}$ — sin AddExpr , MultExpr , PowExpr
- **New:**

```
N = { ..., EqExpr, AddExpr, MultExpr, PowExpr, UnaryExpr, Atom, ... }
```

1.4. Reglas de producción — Nueva jerarquía

La versión antigua tenía EqExpr apuntando directamente a UnaryExpr :

Old:

```
EqExpr ::= UnaryExpr { ( "=" | "<>" | "<" | ">" | "<=" | ">=" | "=>" | "in"
                        " ) UnaryExpr }
```

New — se insertaron tres niveles y el operador :

```
EqExpr ::= AddExpr { ( "=" | "<>" | "<" | ">" | "<=" | ">=" | "≡" |
                    "=>" | "in" ) AddExpr }
AddExpr ::= MultExpr { ( "+" | "-" ) MultExpr }
MultExpr ::= PowExpr { ( "*" | "/" | "%" ) PowExpr }
PowExpr ::= UnaryExpr { "**" UnaryExpr }
```

1.5. Identifier — Corrección de letras mayúsculas

Debido a que la versión antigua no contemplaba las mayúsculas en la definición de **Letter**, los identificadores estaban restringidos. En la nueva versión, con la inclusión de **UpperLetter**, la regla formal $\text{Identifier} ::= \text{Letter} \{ \text{Letter} \mid \text{Number} \mid " \}$ ahora sí permite correctamente el uso de identificadores que inicien o contengan mayúsculas (ver sección 2 para la corrección en Rascal).

1.6. Literales en Atom

La versión antigua ya había añadido **IntLiteral**, **FloatLiteral**, **CharLiteral** a **N** y sus reglas léxicas, pero en la versión **new** se aseguró que **Atom** los referencie explícitamente:

New:

```
Atom ::= Identifier | IntLiteral | FloatLiteral | CharLiteral |
       Application | "(" LogicalExpression ")"
```

1.7. Application — Argumentos opcionales

La corrección se mantiene desde la Entrega_2, ambas versiones coinciden:
Old y New:

```
Application ::= "(" Identifier { LogicalExpression } ")"
```

Los argumentos ahora son opcionales (`{ ... }` en lugar de `{ ... }+`).

2. Regla léxica Id — Solo aceptaba minúsculas

Problema reportado por la TA:

”Your Id lexical rule only accepts lowercase first characters (`[a-z]`), which means standard identifiers like `Set`, `Bool`, or `Nat` won’t parse, changing to `[a-zA-Z]` would fix this immediately.”

Código original (Entrega_2):

```
lexical Id = ([a-z][a-z0-9\-*!>>[a-z0-9\]] \ Reserved;
```

Impacto del error: Cualquier identificador que comenzara con mayúscula (`Set`, `Bool`, `Nat`, `Int`, etc.) generaba un error de parseo, imposibilitando el uso de tipos con nombre capitalizado en todo el lenguaje.

Solución aplicada:

```
lexical Id = ([a-zA-Z][a-zA-Z0-9\-*!>>[a-zA-Z0-9\]] \ Reserved;
```

Se expandió el rango de `[a-z]` a `[a-zA-Z]` en la primera posición y en los caracteres siguientes, ampliando la sintaxis aceptada para permitir identificadores que inicien con cualquier letra. El conjunto `Reserved` permanece intacto — al estar definido con `keyword`, Rascal lo excluye automáticamente del lexema `Id` mediante el operador `\`.

3. Operadores aritméticos y de equivalencia ausentes

Problema reportado por la TA:

”The grammar is missing arithmetic operators (`+`, `-`, `*`, `/`, `%`, `**`) and the equivalence operator `,` which limits what VeriLang programs can express.”

Código original (Entrega_2): La jerarquía de expresiones era: `LogicalExpression` $\rightarrow$ `OrExpr` $\rightarrow$ `AndExpr` $\rightarrow$ `EqExpr` $\rightarrow$ `UnaryExpr` $\rightarrow$ `Atom`. No existían operadores aritméticos ni el operador de equivalencia lógica.

3.1. Nueva jerarquía de expresiones

Se insertaron tres nuevos niveles entre `EqExpr` y `UnaryExpr`: `EqExpr` $\rightarrow$ `AddExpr` $\rightarrow$ `MultExpr` $\rightarrow$ `PowExpr` $\rightarrow$ `UnaryExpr`

EqExpr — ahora referencia `AddExpr` (antes `UnaryExpr`) e incorpora `equiv`:

```

syntax EqExpr
= AddExpr
| eq:    EqExpr "="    AddExpr
| neq:   EqExpr "<>"    AddExpr
| lt:    EqExpr "<"     AddExpr
| gt:    EqExpr ">"     AddExpr
| lte:   EqExpr "<="    AddExpr
| gte:   EqExpr ">="    AddExpr
| equiv: EqExpr "≡"     AddExpr    // <- NUEVO
| impl:  EqExpr "=>"    AddExpr
| inn:   EqExpr "in"    AddExpr
;

```

AddExpr — suma y resta (nuevo):

```

syntax AddExpr
= MultExpr
| add: AddExpr "+" MultExpr
| sub: AddExpr "-" MultExpr
;

```

MultExpr — multiplicación, división, módulo (nuevo):

```

syntax MultExpr
= PowExpr
| mul:    MultExpr "*" PowExpr
| div:    MultExpr "/" PowExpr
| modulo: MultExpr "%" PowExpr
;

```

PowExpr — potencia (nuevo):

```

syntax PowExpr
= UnaryExpr
| pow: PowExpr "**" UnaryExpr
;

```

3.2. Actualización del AST (AST.rsc)

Se añadió el tipo **ArithOp** para representar operaciones aritméticas:

```

data ArithOp
= addOp() | subOp() | mulOp() | divOp() | modOp()
;

```

Se añadió **equivOp()** a **ComparisonOp**:

```

data ComparisonOp
= eqOp() | neqOp() | ltOp() | gtOp() | lteOp() | gteOp()
| equivOp()    // <- NUEVO
| implOp() | inOp()
;

```

Se extendió **Expression** con tres nuevos constructores:

```

data Expression
= ...
| arithmetic(ArithOp, Expression, Expression)    // <- NUEVO
| power(Expression, Expression)                  // <- NUEVO
| literal(Literal)                               // <- NUEVO
;

```

3.3. Actualización del transformador (ToAST.rsc)

Nuevos casos en toExpression:

Producción	Transformación
<AddExpr lhs>+ <MultExpr rhs>	AST::arithmetic(AST::addOp(), ...)
<AddExpr lhs>- <MultExpr rhs>	AST::arithmetic(AST::subOp(), ...)
<MultExpr lhs>* <PowExpr rhs>	AST::arithmetic(AST::mulOp(), ...)
<MultExpr lhs>/ <PowExpr rhs>	AST::arithmetic(AST::divOp(), ...)
<MultExpr lhs>% <PowExpr rhs>	AST::arithmetic(AST::modOp(), ...)
<PowExpr lhs>** <UnaryExpr rhs>	AST::power(...)
<EqExpr lhs> <AddExpr rhs>	AST::comparison(AST::equivOp(), ...)

4. Literales huérfanos — Sin conexión en la gramática

Problema: En la Entrega_2, IntLiteral, FloatLiteral y CharLiteral estaban definidos como lexemas pero ninguna producción de Atom los referencia, por lo que eran sintácticamente inalcanzables.

Solución: Se añadieron como alternativas de Atom:

```

syntax Atom
= atomId: Id
| atomApp: Application
| atomInt: IntLiteral    // <- NUEVO
| atomFloat: FloatLiteral // <- NUEVO
| atomChar: CharLiteral  // <- NUEVO
| paren: "(" LogicalExpression ")"
;

```

Además, se corrigió CharLiteral para aceptar letras mayúsculas:

- Antes: "\' "[a-z] "\' "
- Después: "\' "[a-zA-Z] "\' "

5. Anotaciones @category para resaltado de sintaxis

Problema reportado por la TA (punto menor):

”Minor points: @category annotations should also be added to keywords, identifiers, and literals (not just comments)”

Contexto: En la iteración anterior, la única anotación `@category` existente estaba en los comentarios:

```
lexical WhitespaceAndComment = [\ \t\n\r] | @category="Comment" "#" ![\n]*  
$;
```

Intento de implementación: Se exploró añadir `@category` a `lexical Id`, `lexical IntLiteral`, `keyword Reserved`, etc. Sin embargo, surgieron conflictos con el sistema de palabras reservadas de Rascal. Agregar `@category` a definiciones `keyword` puede interferir con el mecanismo de desambiguación de Rascal, causando que el parser trate incorrectamente las palabras reservadas como identificadores comunes o viceversa.

Estado final: La anotación `@category` en keywords, identifiers y literales **no fue incluida** en la implementación final debido a estos conflictos técnicos con el sistema de resolución de keywords de Rascal. El resaltado de sintaxis se mantiene funcional para comentarios únicamente.

6. Transformación a otro formato

Problema reportado por la TA (punto menor):

.And the code generation stops at AST printing, a transformation to another format would have completed the picture.”

Contexto: En la Entrega_2, el pipeline terminaba imprimiendo el AST. Posteriormente esto se abordó añadiendo:

- **Validación semántica** (`Validate.rsc`) — verifica que operadores y variables referenciadas estén definidos antes de evaluar.
- **Evaluación de expresiones** (`Eval.rsc`, `EvalUtil.rsc`, `EvalExtra.rsc`, `EvalComp.rsc`, `MatchUtil.rsc`, `Val.rsc`) — implementa un evaluador que reduce expresiones a valores concretos (`intVal`, `floatVal`, `boolVal`, `charVal`, `stringVal`, `appVal`), usando *pattern matching* sobre reglas definidas por el usuario. Este es el “otro formato” (valores concretos) al que se transforma el AST generado.

Resumen de archivos modificados

Archivo	Cambios
Syntax.rsc	Id: $[a-z] \rightarrow [a-zA-Z]$; nueva jerarquía <code>AddExpr/MultExpr/PowExpr</code> ; en <code>EqExpr</code> ; <code>Atom</code> incluye <code>IntLiteral</code> , <code>FloatLiteral</code> , <code>CharLiteral</code> ; <code>CharLiteral</code> : $[a-z] \rightarrow [a-zA-Z]$
AST.rsc	Nuevos <code>ArithOp</code> , <code>Literal</code> ; <code>equivOp()</code> ; <code>Expression</code> con <code>arithmetic</code> , <code>power</code> , <code>literal</code> ; <code>operatorDefinition</code> $\rightarrow$ <code>opDefinition</code>
ToAST.rsc	Casos para aritmética, potencia, literales, equivalencia; <code>EqExpr</code> $\rightarrow$ <code>AddExpr</code>
Validate.rsc	Nuevo — validación de operadores y variables existentes
Eval.rsc + rel.	Nuevos — evaluación de expresiones a valores
Val.rsc	Nuevo — tipo <code>Val</code> para representar resultados